DELHI TECHNOLOGICAL UNIVERSITY

Department of Computer Science & Engineering

FINAL YEAR PROJECT
DETAILED RESEARCH PLAN

Project Title

Self-Supervised Learning for Medical Image Classification Using Labelled and Unlabelled Data

Academic Year	2026-27
Group ID	DTU/CSE/BTP/GP053
Guide	Dr. Ravin Ahuja
Group Members	Manish Singh (23/CS/244) Shivam (23/CS/394) Siddharth Kumar Maurya (23/CS/411)
Junior Team Member	Saurav Kumar (25/CS/374)

TABLE OF CONTENTS

S. No.	Content	Page No.
1.	Abstract	3
2.	Problem Statement	4
3.	Motivation	4
4.	Objectives	4
5.	Brief Literature Review	4
6.	Research Gap	6
7.	Proposed Approach	7
8.	Self-Supervised Learning Pipeline	8
9.	Random Masking and Two-View Generation	9
10.	CNN Representation Learning	10
11.	Pre-training and Fine-Tuning	11
12.	Baseline Model	12
13.	Dataset and Data Preparation	12
14.	Experimental Design	13
15.	Evaluation Metrics	15
16.	Comparative / Ablation Study	16
17.	Explainability	16
18.	Technical Requirements	17
19.	Expected Outcomes	17
20.	Limitations	18
21.	Contribution / Impact	18
22.	Work Plan	19
23.	Gantt Chart	20
24.	Expected Deliverables	20
25.	References	20

1. Abstract

Medical image classification is an important application of deep learning in healthcare. Convolutional Neural Networks (CNNs) have shown strong performance in learning visual patterns from medical images. However, training an effective CNN generally requires a sufficiently large amount of labelled data. In medical imaging, obtaining labelled data can be difficult because image annotation often requires expert knowledge and significant time. At the same time, large collections of unlabelled medical images may be available. This creates a need for learning methods that can make effective use of both labelled and unlabelled data.

This project investigates **Self-Supervised Learning for Medical Image Classification Using Labelled and Unlabelled Data**. The main objective is to study whether useful visual representations can be learned from unlabelled medical images before using labelled data for classification. The proposed approach uses a CNN-based self-supervised learning framework with **random masking**. During self-supervised pre-training, an unlabelled medical image is transformed into two different masked views. Different regions of the same image are randomly hidden to create these views. Both views are then passed through the same CNN encoder with shared weights. The CNN produces feature representations for the two views, and a consistency loss is used to encourage their representations to remain similar. The difference between the representations provides a learning signal without requiring class labels. Through this process, the CNN is expected to learn useful visual features from unlabelled images.

After self-supervised pre-training, the learned CNN representation is used for supervised fine-tuning with labelled medical images. A classification layer is added to perform the required medical image classification task. The performance of the proposed approach will be compared with a conventional CNN trained using labelled data. This comparison will help determine whether self-supervised pre-training provides an advantage when the amount of labelled data is limited.

The experimental study will use suitable datasets from **MedMNIST**, which provides standardized medical image datasets covering different modalities and classification tasks. The project will investigate the effect of using unlabelled data during pre-training and will evaluate the resulting classification performance using appropriate metrics such as **accuracy (ACC)** and **area under the ROC curve (AUC)**. Additional analysis may include confusion matrices and visual explanation methods such as **Grad-CAM**, where appropriate, to understand the regions influencing model predictions.

The project will also study the contribution of important components of the proposed approach, particularly random masking and self-supervised pre-training. The experiments will help provide a better understanding of how unlabelled medical images can contribute to representation learning and whether this can reduce dependence on large amounts of labelled data.

The expected outcome of this project is a practical experimental analysis of a CNN-based self-supervised learning approach for medical image classification. It focuses on understanding the potential of self-supervised representation learning in a controlled research setting. The work is aligned with **Sustainable Development Goal 3 (SDG 3): Good Health and Well-being**, through research into machine learning methods that may support more effective processing and analysis of medical images.

2. Problem Statement

Medical image classification is difficult because images from different classes can look similar. Only a limited number of medical images may have reliable labels. Many other images can remain unlabelled. The problem is to learn useful visual patterns from both sources and improve classification when labelled data is limited.

3. Motivation

Labelling medical images requires time and expert effort. This makes it difficult to build large labelled datasets for every medical imaging task. Unlabelled images are easier to collect, but a normal supervised classifier cannot directly use them because the training signal depends on class labels.

Self-supervised learning offers a way to learn from images without requiring manual labels for every pre-training example. The model creates its own learning signal from different views of the same image. In this project, random masking provides the change between the views. The aim at this stage is not to diagnose anything. The aim is to learn image features that remain useful when part of an image is hidden.

The project is also motivated by the need for a practical student-level experiment. The approved synopsis uses a lightweight CNN, rather than a large model. This makes it possible to study the effect of unlabelled data with limited compute and with clear baseline comparisons.

4. Objectives

- Learn useful visual representations from unlabelled medical images using self-supervised learning.
- Use random masking to create two views of the same unlabelled image.
- Pass both views through the same CNN so that the encoder learns shared visual features.
- Use a consistency loss to reduce disagreement between the two learned representations.
- Fine-tune the pre-trained CNN with labelled medical images for classification.
- Compare the proposed approach with a normal supervised CNN trained with the same labelled data conditions.
- Evaluate the models using accuracy, AUC, precision, recall, F1-score, and confusion matrices as appropriate for the selected datasets.
- Use Grad-CAM based explanation maps where supported by the final CNN to visualize image regions that influence predictions.

The objectives remain within the approved synopsis. They do not introduce transformers, language models, generative systems, or unrelated datasets.

5. Brief Literature Review

The literature review covers the MedMNIST benchmark and selected work on self-supervised learning, including methods developed for medical images. These studies provide context for the project and help define a practical question that can be tested with a lightweight CNN. Results reported in previous papers are distinguished from results that will be obtained in this project.

5.1 MedMNIST v2 Benchmark

Yang et al. introduced MedMNIST v2 as a lightweight benchmark containing 12 standardized 2D datasets and 6 standardized 3D datasets. The images are reduced to 28 x 28 for 2D data and 28 x 28 x 28 for 3D data. The benchmark covers binary, multi-class, ordinal, and multi-label tasks. This makes it useful for controlled classification experiments where the focus is on the learning method rather than on handling very large images. The project uses selected MedMNIST datasets for this reason [1].

5.2 Masked Siamese Networks for Label-Efficient Learning

Assran et al. present Masked Siamese Networks (MSN), a self-supervised framework that matches representations of a masked view with another view of the same image. The paper shows that masking can support label-efficient image representation learning. The model is built around a Siamese-style representation learning setting and is demonstrated mainly with large-scale vision backbones. For the current project, the important conceptual link is that different views of the same image can provide a learning signal without requiring a class label [2].

5.3 Self-supervised Learning for Medical Image Analysis: Discriminative, Restorative, or Adversarial?

Haghighi et al. look at three kinds of self-supervision used in medical image analysis (discriminative, restorative, and adversarial) and propose DiRA, a framework that combines all three. Their results show that each kind picks up different information, so medical-image SSL is a family of approaches with different learning signals and trade-offs rather than one method. For this project, that is a reason to keep the method definition narrow: the planned work uses view consistency after random masking only, and does not try to combine several SSL paradigms [3].

5.4 Self-supervised Learning Framework Application for Medical Image Analysis: A Review and Summary

Zeng et al. review medical-image SSL studies up to 2024. The review notes that manual annotation is labour-intensive and that SSL can learn representations from unlabelled data before fine-tuning on a smaller labelled set. It also shows the wide use of SSL across CT, MRI, X-ray, histology, and ultrasound tasks. This review supports the central motivation of using unlabelled medical images as a pre-training source [4].

5.5 CoBooM: Codebook Guided Bootstrapping for Medical Image Representation Learning

Singh and Mishra propose CoBooM at MICCAI 2024. The method uses codebooks to combine continuous and discrete representations for medical image SSL. The paper reports experiments on chest X-ray and fundus datasets. It demonstrates the continued movement toward specialized representation learning methods in medical imaging. In the present project, this work is used only as recent context. The B.Tech implementation intentionally remains simpler and CNN-based so that the effect of unlabelled pre-training and masking can be studied with manageable computation [5].

Table 1. Literature review summary

Ref.	Year / Venue	Main idea	Relation to project
[1]	2023 / Scientific Data	MedMNIST v2 benchmark with 2D/3D biomedical datasets.	Provides the evaluation benchmark and official dataset splits.
[2]	2022 / ECCV	Masked Siamese representation learning.	Supports the idea of learning from different masked views without class labels.
[3]	2024 / Medical Image Analysis	DiRA: combines discriminative, restorative and adversarial SSL.	Places the project in the broader medical-image SSL landscape.
[4]	2024 / BioMedical Engineering Online	Review of SSL methods across medical imaging tasks.	Supports the motivation for label-efficient pre-training.
[5]	2024 / MICCAI	Codebook-guided medical image representation learning.	Recent context showing specialized SSL; motivates a lightweight controlled CNN study.

6. Research Gap

Earlier work shows that self-supervised learning can use unlabelled images to learn representations before fine-tuning. Medical-image research already covers many approaches, including different learning objectives and specialized representation mechanisms [2]-[4]. Our question is simpler: does a lightweight CNN benefit from random-masking based pre-training when fewer labelled images are available?

The gap we address is a practical one. We study a single simple CNN pipeline that uses unlabelled MedMNIST images for pre-training and compare it with a supervised baseline under the same labelled-data conditions, which keeps the question easy to explain and evaluate. We want to see whether the approved pipeline gives a measurable benefit, and at which labelled-data settings that benefit is clearest.

7. Proposed Approach

The proposed approach follows the approved synopsis. The complete flow is shown below.

Figure 1. Overall Project Workflow

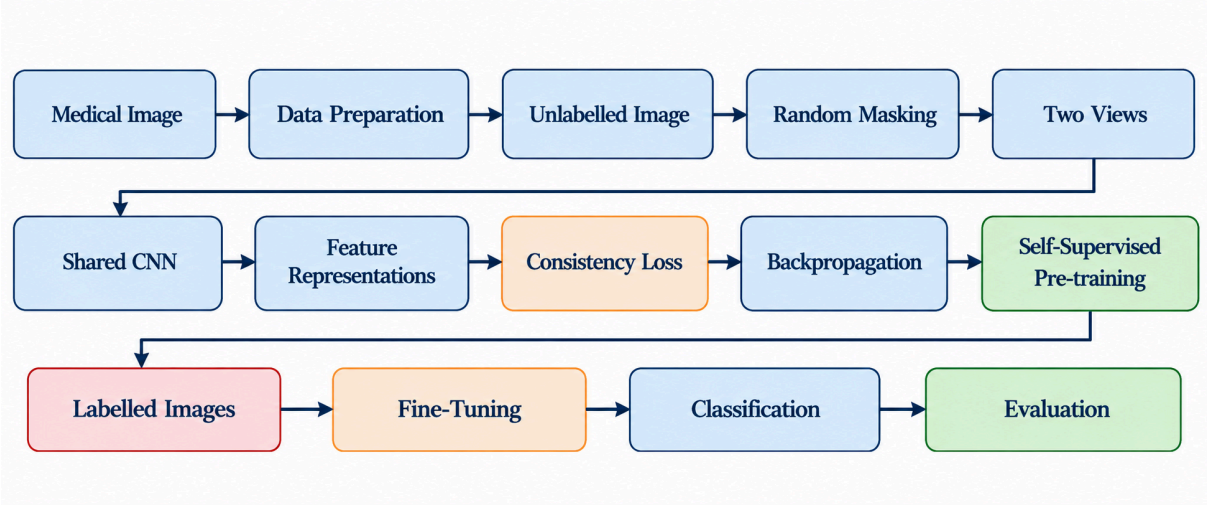


Figure 1 lays out the whole project as one chain of steps, read left to right over three rows. In the first row, a medical image goes through data preparation, is treated as an unlabelled image, and is randomly masked to produce two views. In the second row, the views pass through the shared CNN to give feature representations, the consistency loss is computed, and backpropagation updates the network. That completes self-supervised pre-training (the green box). The third row brings in labelled images for fine-tuning, followed by classification and evaluation. The key difference from ordinary supervised learning is that the encoder learns from unlabelled images first, and labels only come in at the fine-tuning step.

7.1 Why self-supervised pre-training is used

A label tells the classifier which class an image belongs to. It is useful for the final prediction task. However, a model can also learn useful visual structure before it knows the class names. Self-supervised learning creates a learning signal from the image itself. In this project, the signal comes from asking the CNN to produce consistent representations for two masked views of the same image.

7.2 Intuitive explanation of the learning signal

Imagine covering different parts of the same image twice. The two views are different at the pixel level, but they still come from the same original image. A useful visual representation should not change completely just because some pixels are hidden. The CNN is therefore encouraged to produce similar representations for both views. Over many examples, this pressure can teach the encoder to use broader image information.

8. Self-Supervised Learning Pipeline

Figure 2. Self-Supervised Learning Architecture

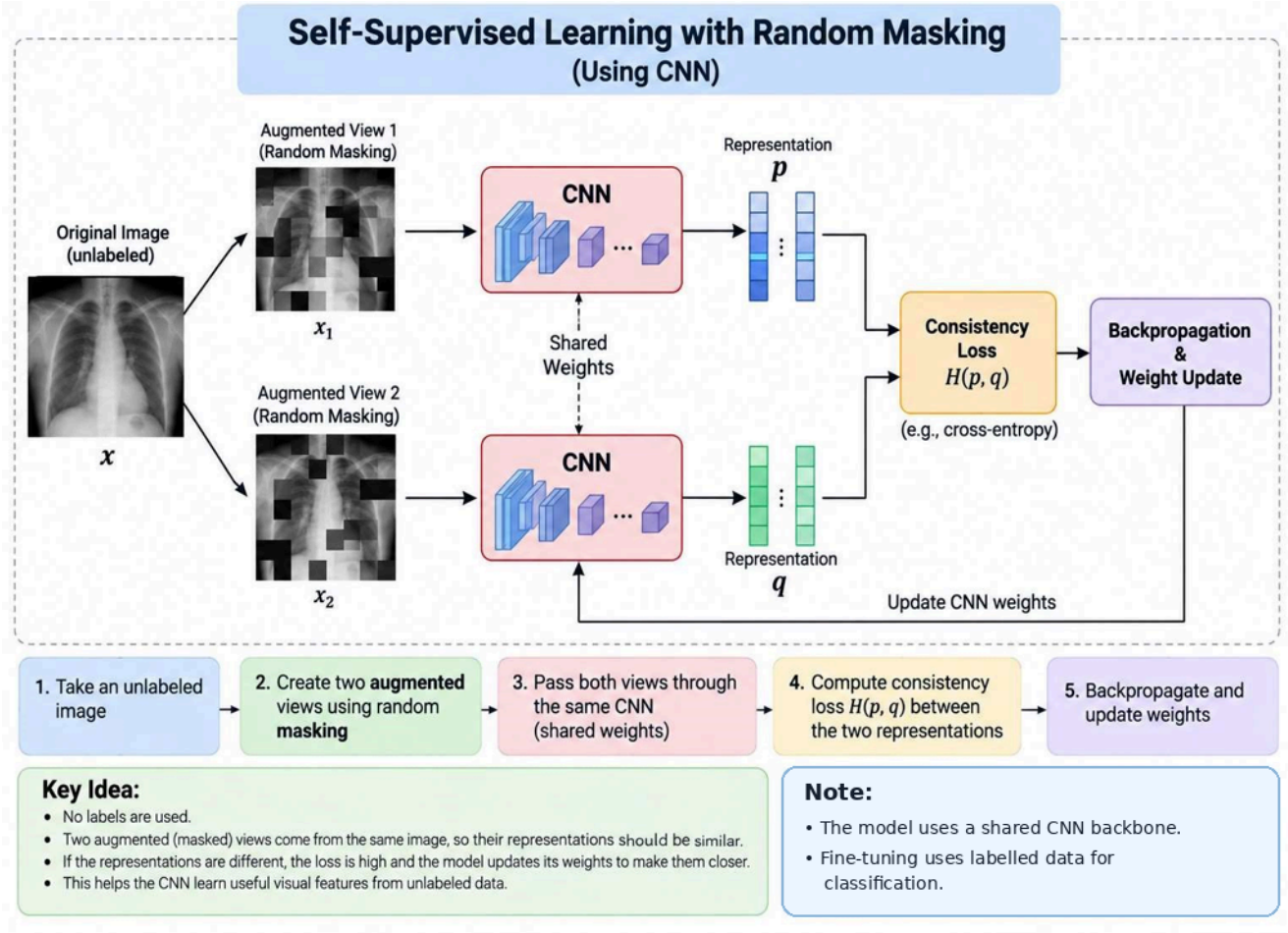


Figure 2 zooms in on the self-supervised stage, using a chest X-ray as the example. The original unlabelled image x is turned into two randomly masked views, x1 and x2, and each one goes into a CNN. The dotted line marked shared weights shows that the two CNNs are really the same network. They produce the representations p and q, the consistency loss H(p, q) compares them, and backpropagation updates the CNN weights, as the arrow looping back to the CNNs shows. The figure names cross-entropy as one example of the loss, but the actual choice will be made during implementation. The two boxes at the bottom repeat the key points: no labels are used here, and labelled data only comes in later for fine-tuning.

8.1 Input image

We start with an unlabelled medical image **x** from the training set. Its class label is ignored during pre-training, so the image is only the starting point for making two views.

8.2 Two masked views

Random masking turns **x** into two different views, $\mathbf{x}_1$ and $\mathbf{x}_2$. Each view hides different regions, so the two views show different parts of the same image. Section 9 explains the masking in more detail.

8.3 Shared CNN and feature representations

Both views go through the same CNN. This means one encoder with one set of weights, not two separate networks. For each view it gives a feature representation, called **p** for the first view and **q** for the second.

8.4 Consistency loss

The consistency loss **H(p,q)** measures how different p and q are. Since both views come from the same image, the aim is to push the two representations close together. We will choose the exact loss function during implementation and keep it the same in all the main experiments.

There is one risk with this kind of loss. If the only goal is to make the two representations match, the CNN can cheat by giving the same output for every image, and the loss drops to zero without anything useful being learned (representation collapse). To avoid this, we plan to add a small predictor layer on one branch and stop the gradient on the other branch, as done in SimSiam and BYOL style methods. We will also watch the spread of the representations during pre-training to catch collapse early. The predictor layer is a small training detail and is not drawn in Figures 2 and 4. If collapse still happens, we will switch to a contrastive loss that uses negative pairs.

8.5 Weight update

Backpropagation calculates the gradients of this loss and the optimizer updates the shared CNN weights. This is repeated over the unlabelled images in the training split, with two masked views generated for each image. No class labels are needed at any point.

8.6 Moving to fine-tuning

Once pre-training is finished, the learned weights are used to start the classifier. A classification layer is added and the model is fine-tuned on labelled images (see Section 11), then compared with a supervised CNN baseline.

9. Random Masking and Two-View Generation

Figure 3. Random Masking and Two-View Generation

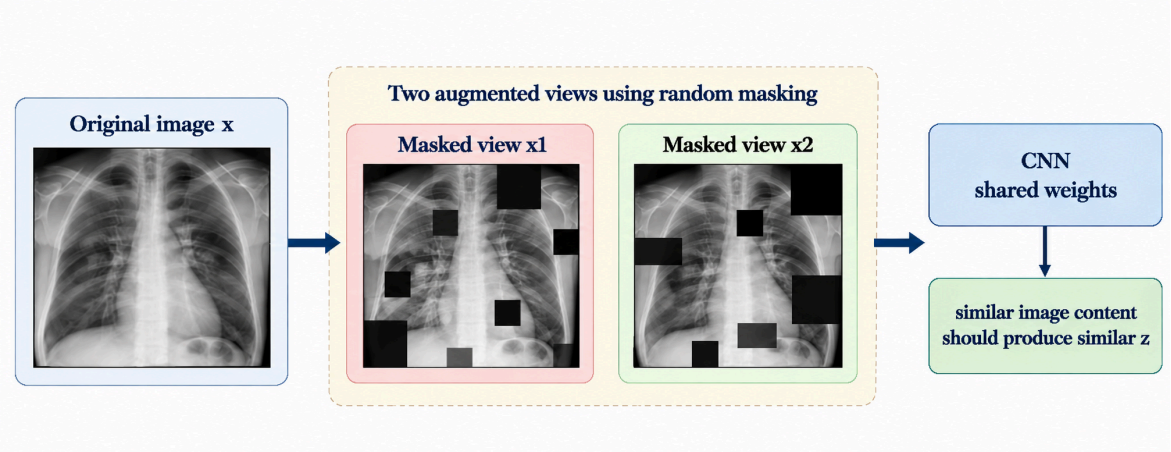


Figure 3 shows the masking step on a chest X-ray. The original image x on the left gives two views, x1 and x2. In each one, black squares cover different parts of the image, so the visible pattern changes from one view to the other even though both come from the same X-ray. Both views then go into the CNN with shared weights, and the box on the right states the goal: similar

image content should give similar representations (written as z in this figure, which is the same idea as p and q elsewhere in this plan). Since the hidden patches land in different places each time, the model cannot depend on any single region and has to learn features that hold up despite the masking.

9.1 How the masking works

Random masking hides some regions of the image, using patches or blocks. The hidden locations are different in the two views, so the CNN sees different visible parts of the same image. This needs no label, so it works directly on unlabelled data.

9.2 Why two views are used

Both views come from the same image, so a good representation should stay similar for both even though some pixels are hidden. This stops the CNN from depending too much on one visible region and pushes it towards features that cover the whole image.

9.3 Practical masking plan

The masking rate and patch size will be chosen during implementation, starting with initial settings tested on the selected MedMNIST dataset. If time and computing resources allow, we will run a small experiment comparing different masking settings.

10. CNN Representation Learning

A CNN learns visual patterns by sliding filters over an image. Early layers pick up simple local patterns such as edges or changes in intensity, and deeper layers combine them into more useful features. In this project the CNN is the shared encoder for both masked views.

10.1 Learned representations

Each masked view is turned into a numerical feature vector (p and q in the workflow). Because both views go through the same encoder, these vectors can be compared directly, and the consistency objective pushes the CNN to give similar vectors for views of the same image. This is how it learns useful features from unlabelled images.

10.2 Using the learned weights

After pre-training, the encoder weights are used to initialize the classification model. A classification layer is then trained on labelled images so the learned features can be used for the actual task.

10.3 Judging the representations

We will judge them by comparing the fine-tuned model with a supervised CNN trained under the same labelled-data conditions. This shows whether pre-training helps when fewer labelled images are available.

11. Pre-training and Fine-Tuning

Figure 4. Pre-training and Fine-Tuning Pipeline

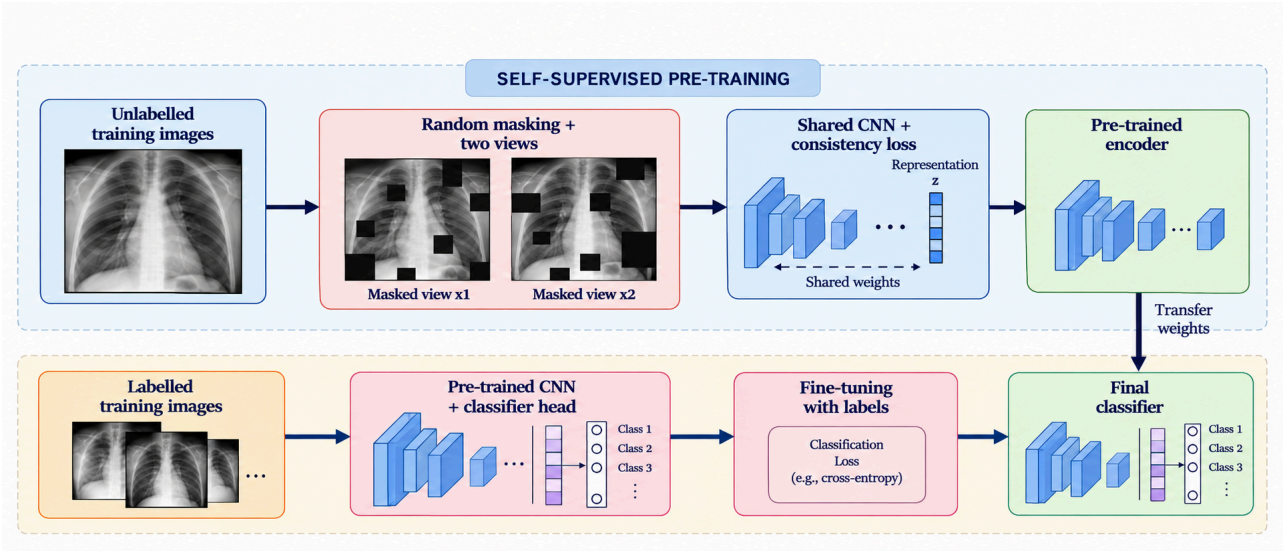


Figure 4 splits the project into two stages. The top band is self-supervised pre-training: unlabelled training images are masked into two views, passed through the shared CNN with the consistency loss, and the result is a pre-trained encoder. The arrow marked transfer weights carries that encoder into the bottom band. There, a classifier head is attached to the pre-trained CNN and the model is fine-tuned on labelled training images using a classification loss such as cross-entropy, which gives the final classifier. Labels only appear in the bottom band.

11.1 Stage 1: pre-training

The first stage is the self-supervised pre-training described in Section 8. It uses only unlabelled training images and gives us a pre-trained encoder.

11.2 Transferring the weights

The learned weights are then copied into the classification model. They give the encoder a starting set of visual features instead of random initialization, and a classification head is attached to predict the target classes.

11.3 Stage 2: fine-tuning with labels

In the second stage the model is trained on labelled medical images. A classification loss such as cross-entropy compares the predictions with the true labels, and backpropagation with an optimizer updates the parameters of the CNN and the classification head.

11.4 Final classifier

After fine-tuning, the model predicts the target classes. It combines the features learned during pre-training with the class information from the labelled data, and will be evaluated on the selected MedMNIST datasets using the metrics in Section 15. The test set stays separate from training and model selection.

11.5 Comparison with a supervised CNN

We will compare the model with a conventional supervised CNN trained on labelled data only, using the same dataset splits and the same labelled-data fractions. This shows whether pre-training on unlabelled images improves classification, especially when labelled data is limited.

12. Baseline Model

The baseline is a normal supervised CNN. It will use the same selected MedMNIST datasets and the same data splits as the self-supervised model. The main difference is that the baseline starts from random initialization and learns only from labelled data. The exact CNN, for example a small network of a few convolutional blocks or a ResNet-18 style network adapted to 28 x 28 inputs, will be fixed before any experiment and used unchanged for both models.

Table 2. Baseline versus proposed model

Aspect	Supervised CNN baseline	Self-supervised CNN	Purpose of comparison
Initialization	Random initialization	Pre-trained on unlabelled images	Tests whether pre-training helps.
Unlabelled data	Not used for learning signal	Used during pre-training	Tests value of unlabelled images.
Labelled data	Used for complete training	Used mainly for fine-tuning	Keeps final task supervised.
Architecture	Small CNN	Same CNN encoder	Reduces architecture-related confounding.
Evaluation	Same test set	Same test set	Allows direct comparison.

The aim of the baseline is not to prove that self-supervised learning must always be better. It gives the project a clear reference point. A result where the two methods are similar is also informative.

13. Dataset and Data Preparation

The project uses MedMNIST as the benchmark family. MedMNIST v2 provides 12 standardized 2D datasets and 6 standardized 3D datasets, with small image resolutions and different classification tasks [1]. The benchmark paper also describes the official dataset splits.

To keep the B.Tech experiment manageable, the initial evaluation can use a small 2D subset. A practical subset is PathMNIST, DermaMNIST, and PneumoniaMNIST. This recommendation is a planning choice, not a claim that the approved synopsis restricts the project to these three datasets.

Table 3. Candidate initial MedMNIST 2D subset

Dataset	Task	Classes	Total samples	Train / Val / Test	Resolution
PathMNIST	Multi-class	9	107,180	89,996 / 10,004 / 7,180	3 x 28 x 28
DermaMNIST	Multi-class	7	10,015	7,007 / 1,003 / 2,005	3 x 28 x 28
PneumoniaMNIST	Binary	2	5,856	4,708 / 524 / 624	1 x 28 x 28

The dataset counts and split information in Table 3 follow the MedMNIST v2 benchmark description [1]. The project will use the official train, validation, and test splits. Unlabelled pre-training data will come only from the training portion so that the test set remains separate.

13.1 Data preparation steps

- Load the selected MedMNIST dataset using the approved MedMNIST and torchvision/PyTorch tooling.
- Convert images to tensors and apply the required normalization.
- Keep the official train, validation, and test partitions separate.
- Create the unlabelled pre-training view from training images without using class labels.
- Create the labelled fine-tuning subsets from the same training split, sampled with the same class proportions, and use the same subset for both the baseline and the SSL model.
- Use the validation set for model selection and parameter decisions.
- Use the test set only for final reporting after all choices are fixed.

13.2 Handling different task types

The selected initial subset contains binary and multi-class classification tasks. This avoids mixing multi-label and ordinal tasks in the first controlled experiment. If additional MedMNIST datasets are added later, the loss and metric definitions will be adjusted to the specific task type rather than applying one metric blindly to every dataset.

13.3 Dataset caveats

The three datasets are not equally easy to work with. DermaMNIST is strongly imbalanced, with one class making up most of the images, and PneumoniaMNIST has more pneumonia than normal cases. On such data accuracy alone can look good even when the minority classes are handled badly, so macro-averaged F1 and AUC will be reported next to accuracy.

Also, the official test set of PathMNIST comes from a different clinical source than its training data [1], and PneumoniaMNIST may show a similar gap between validation and test scores. A drop on the test set may therefore come from this shift and not from the method itself, so the results will be read with that in mind.

14. Experimental Design

The experiment is designed around one main question: does self-supervised pre-training with random masking provide a useful representation when fewer labels are available? The baseline and proposed model will use the same selected datasets and the same evaluation splits.

14.1 Main experimental groups

- Supervised baseline CNN using 100 percent of the available training labels.
- Supervised baseline CNN using reduced label settings such as 10 percent, 25 percent, and 50 percent where feasible.
- Self-supervised CNN pre-trained on the unlabelled training images and then fine-tuned with the same label fractions.
- Optional ablation in which the self-supervised stage is run without random masking to isolate the role of the masking operation.
- Optional linear probe in which the pre-trained encoder is frozen and only the classification layer is trained, to check the quality of the learned representations directly.

14.2 Training sequence

1. Prepare the selected dataset and verify the official split.
2. Train the supervised baseline.
3. Implement the random-masking view generator.
4. Train the CNN with the consistency objective on unlabelled training images.
5. Transfer the learned CNN weights to the classifier and fine-tune with labelled data.
6. Evaluate on the test set.
7. Repeat under reduced label settings and with three random seeds.
8. Run the masking ablation if resources allow.
9. Generate confusion matrices and Grad-CAM maps for representative samples.

14.3 Fair comparison rules

- Use the same selected dataset and same test set for both methods.
- Use the same labelled subset sizes for baseline and SSL fine-tuning.
- Do not use test labels for model selection.
- Keep the main CNN architecture unchanged between the baseline and SSL model.
- Report all experiments using the same metric definitions.
- Repeat every main experiment with three random seeds and report the mean and standard deviation, using the same seeds for the baseline and the SSL model.
- Give both models the same fine-tuning budget (number of epochs and optimizer settings), with learning rate and early stopping chosen on the validation set only.
- Apply the same basic preprocessing to both models, and if any extra augmentation is used for one model it will also be used for the other.

One difference is deliberate: the SSL model learns from all the unlabelled training images before fine-tuning, while the baseline cannot use them. Testing the value of that extra unlabelled data is the whole point of the project, so this is not an unfair advantage. The labelled images used for training are the same for both models.

14.4 Experimental Evaluation Workflow

Figure 5. Experimental Evaluation Workflow

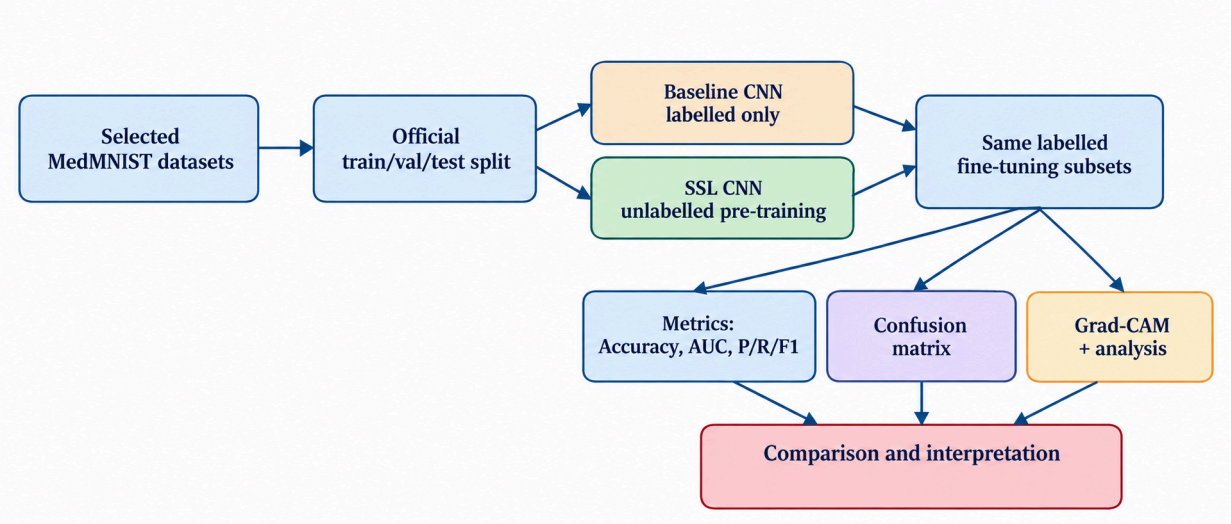


Figure 5 shows how the experiments are kept comparable. Each selected MedMNIST dataset is divided using its official train, validation, and test split. The baseline CNN (labelled data only) and the SSL CNN (unlabelled pre-training first) are then both trained on the same labelled fine-tuning subsets. Their results are looked at in three ways: the metrics (accuracy, AUC, precision, recall, F1), confusion matrices, and Grad-CAM maps with analysis. All three feed into the final comparison and interpretation. The figure does not draw the test set as its own box, but as described in Section 14.3, both models are evaluated on the same official test set.

15. Evaluation Metrics

Table 4. Evaluation metrics and interpretation

Metric	What it measures	Use in this project	Caution
Accuracy	Fraction of correct predictions.	Binary and multi-class tasks.	Can hide poor performance on imbalanced classes.
AUC	Ranking ability across positive and negative cases.	Binary tasks and suitable one-vs-rest / macro settings for multi-class evaluation.	Definition and averaging must match the task.
Precision	How many predicted positives are correct.	Binary and per-class analysis.	Sensitive to false positives.
Recall	How many actual positives are detected.	Binary and per-class analysis.	Sensitive to false negatives.
F1-score	Balance between precision and recall.	Binary and macro/weighted multi-class reporting.	Should be reported with the averaging rule.

Metric	What it measures	Use in this project	Caution
Confusion matrix	Counts of predictions by true and predicted class.	Multi-class error analysis.	Not a single-number performance measure.

The evaluation will report AUC and accuracy, along with precision, recall, and F1-score as specified in the approved synopsis. Metric definitions and averaging methods will be chosen to match each dataset's task.

For the imbalanced datasets, macro-averaged F1 and AUC will be given the same weight as accuracy when comparing the two models.

16. Comparative / Ablation Study

The main comparison is between a normal supervised CNN and the same CNN after self-supervised pre-training. A small ablation is planned to test whether random masking itself contributes to the learned representation.

Table 5. Planned comparison matrix

Experiment	Unlabelled pre-training	Random masking	Label fraction	Purpose
A. Supervised baseline	No	No	100%, 50%, 25%, 10% if feasible	Reference performance under limited labels.
B. SSL + masking	Yes	Yes	Same label fractions	Main proposed method.
C. SSL without masking (optional)	Yes	No	Same label fraction	Ablation for the masking component.
D. Mask-ratio sensitivity (optional)	Yes	Yes - varied	One label fraction	Check whether masking strength changes the result.

The main purpose is to isolate the value of the self-supervised pre-training stage and, if resources allow, the value of random masking.

17. Explainability

The approved tool list includes pytorch-grad-cam. Grad-CAM can be used to produce a heat map showing image regions that contribute to a CNN prediction. The project will use such maps for representative correct and incorrect predictions.

The heat map is only an explanation aid. It does not prove that the model is correct. It also does not establish clinical interpretability or medical reliability. The project will therefore use Grad-CAM to inspect model behaviour.

MedMNIST images are only 28 x 28 pixels, so the final feature maps are very small and the Grad-CAM heat maps will be coarse. They are expected to show rough regions and not fine detail. To make the comparison useful, maps for the baseline and the SSL model will be produced for the same test images.

18. Technical Requirements

Table 6. Software requirements

Category	Tool / library	Planned use
Programming language	Python	Implementation and experiment scripts.
Deep learning	PyTorch	CNN training, self-supervised pre-training, fine-tuning.
Computer vision utilities	torchvision	Tensor transforms, data loading, augmentation utilities.
Dataset	MedMNIST	Biomedical image benchmark.
Numerical computing	NumPy	Array operations and dataset handling.
Evaluation	scikit-learn	Metrics, confusion matrices, and analysis utilities.
Explainability	pytorch-grad-cam	Grad-CAM explanation maps.
Plotting	Matplotlib, Seaborn	Learning curves, confusion matrices, result plots.

Table 7. Hardware and software-compute requirements

Level	Requirement	Reason
Minimum	Modern multi-core CPU, 16 GB RAM, 4 GB GPU VRAM preferred	Allows small-batch training and basic experimentation. CPU-only runs are possible but slower.
Preferred	Modern i5/i7/Ryzen 5/7 class CPU, 24-32 GB RAM, 6-8 GB CUDA-capable GPU	More practical for repeated pre-training, fine-tuning, and multiple label settings.
Storage	At least 20-30 GB free working space	Dataset files, cached weights, logs, and generated figures.
Software compute	PyTorch with CUDA when a supported NVIDIA GPU is available	Reduces training time for repeated CNN experiments.

The project will use the available computing hardware and manageable batch sizes. The final report will record the hardware and training settings actually used.

19. Expected Outcomes

- A CNN model pre-trained on unlabelled medical images using the planned self-supervised masking procedure.
- A fine-tuned classification model trained with labelled medical images.
- Classification results on the selected MedMNIST datasets.

- Accuracy, AUC, precision, recall, and F1-score reported with task-appropriate definitions.
- Confusion matrices for the classification tasks where applicable.
- A comparison between the normal supervised CNN and the self-supervised CNN.
- Sample predictions with predicted class labels.
- Grad-CAM explanation maps for representative model predictions.
- A clear analysis of whether reduced labels change the relative benefit of self-supervised pre-training.
- Results reported as mean and standard deviation over three random seeds, including cases where pre-training gives no clear gain.

20. Limitations

- MedMNIST images are small and standardized. Performance on this benchmark may not represent performance on high-resolution clinical images.
- Only a subset of MedMNIST may be used because of time and compute limits.
- Limited labelled data can make model comparison sensitive to the selected label fraction and random seed.
- Class imbalance may affect accuracy and other metrics on some datasets.
- The proposed pipeline uses a lightweight CNN and may not represent the full range of recent medical-image SSL architectures.
- Self-supervised training can collapse to a useless constant output. The planned predictor layer and stop-gradient reduce this risk, but it will still need to be checked during pre-training.
- Grad-CAM maps on 28 x 28 images are coarse and give only a rough idea of where the model is looking.
- Some test sets, such as PathMNIST, come from a different source than the training data, so test results can drop for reasons that have nothing to do with the method.
- The project uses benchmark datasets and should not be interpreted as clinical deployment or medical-grade validation.

21. Contribution / Impact

The main contribution of the project is a focused study of whether unlabelled medical images can help a lightweight CNN learn useful representations before classification when labelled data is limited. The project also provides a controlled comparison between supervised learning and self-supervised pre-training on selected MedMNIST datasets.

The work aligns with SDG 3 - Good Health and Well-being because it studies a machine learning problem involving medical images. The alignment is educational and research-oriented.

22. Work Plan

Table 8. Work plan and expected outputs

Phase	Planned period	Main activities	Expected output
1. Literature study	Early Aug 2026	Read the DTU requirements, the approved synopsis, and the literature on MedMNIST and self-supervised learning.	Finalized research question and literature notes.
2. Dataset preparation	Aug 2026	Set up MedMNIST loading, split checks, transforms, and basic visualization.	Verified data pipeline.
3. Baseline CNN	Mid Aug-early Sep 2026	Implement and train supervised CNN on the selected datasets.	Baseline models and initial metrics.
4. Masking + SSL pipeline	Late Aug-mid Sep 2026	Implement two-view random masking, shared CNN, representation output, and consistency loss.	Working self-supervised training pipeline.
5. Pre-training	Sep 2026	Run unlabelled pre-training and record training behaviour.	Pre-trained CNN weights and logs.
6. Fine-tuning	Late Sep-early Oct 2026	Fine-tune using selected label fractions.	Classification models and validation results.
7. Evaluation + ablation	Oct 2026	Compute metrics, confusion matrices, optional no-mask ablation, compare models, and run the final tests on the code.	Comparison tables and plots.
8. Explainability	Oct 2026	Generate Grad-CAM maps and inspect representative predictions.	Explanation figures and analysis.
9. Report + presentation	Late Oct-mid Nov 2026	Analyze findings, write the final report, and prepare the presentation.	Final report and presentation.

23. Gantt Chart

Figure 6. Gantt Chart

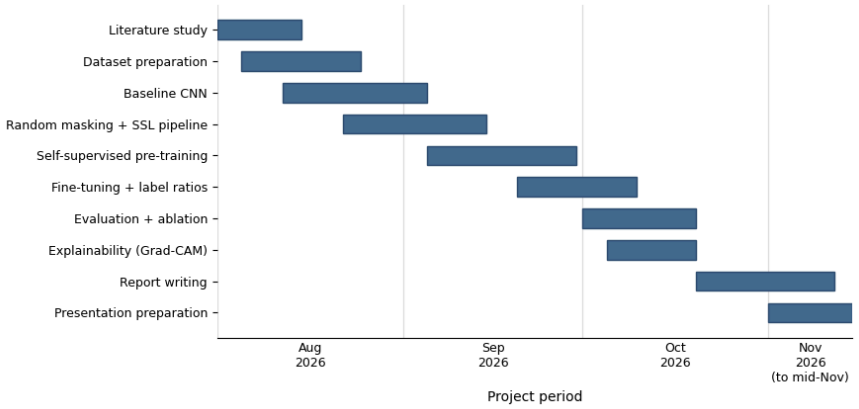


Figure 6 is the Gantt chart for the work plan in Table 8, running from August to mid-November 2026. The literature study and dataset preparation come first in August, with the baseline CNN and the masking and SSL pipeline starting right after. Pre-training runs through September, and fine-tuning, the evaluation with the ablation, and the Grad-CAM work finish by the end of October, so all the main technical work sits in the first part of the timeline. The last weeks are only for report writing and the presentation, which end in mid-November.

24. Expected Deliverables

- Clean project code for data preparation, CNN baseline, self-supervised pre-training, fine-tuning, evaluation, and visualization.
- Trained model checkpoints for the main experiments.
- Dataset and experiment configuration notes.
- Metric tables, confusion matrices, learning curves, and representative Grad-CAM maps.
- Comparison of supervised and self-supervised models under matched label fractions.
- Final project report and presentation.

25. References

[1] J. Yang, R. Shi, D. Wei, Z. Liu, L. Zhao, B. Ke, H. Pfister, and B. Ni, "MedMNIST v2 - A large-scale lightweight benchmark for 2D and 3D biomedical image classification," Scientific Data, vol. 10, art. 41, 2023, doi:10.1038/s41597-022-01721-8.

[2] M. Assran, M. Caron, I. Misra, P. Bojanowski, F. Bordes, P. Vincent, A. Joulin, M. Rabbat, and N. Ballas, "Masked Siamese Networks for Label-Efficient Learning," in Proc. European Conference on Computer Vision (ECCV), 2022, doi:10.1007/978-3-031-19821-2_26.

[3] F. Haghighi, M. R. Hosseinzadeh Taher, M. B. Gotway, and J. Liang, "Self-supervised Learning for Medical Image Analysis: Discriminative, Restorative, or Adversarial?," Medical Image Analysis, vol. 94, art. 103086, 2024, doi:10.1016/j.media.2024.103086.

[4] X. Zeng, N. Abdullah, and P. Sumari, "Self-supervised learning framework application for medical image analysis: a review and summary," BioMedical Engineering OnLine, vol. 23, art. 107, 2024, doi:10.1186/s12938-024-01299-9.

[5] A. Singh and D. Mishra, "CoBooM: Codebook Guided Bootstrapping for Medical Image Representation Learning," in Medical Image Computing and Computer-Assisted Intervention (MICCAI), 2024, LNCS 15012, pp. 23-33, doi:10.1007/978-3-031-72390-2_3.